

TigellAI — un Agente AI con n8n



DataMasters

Chi sono



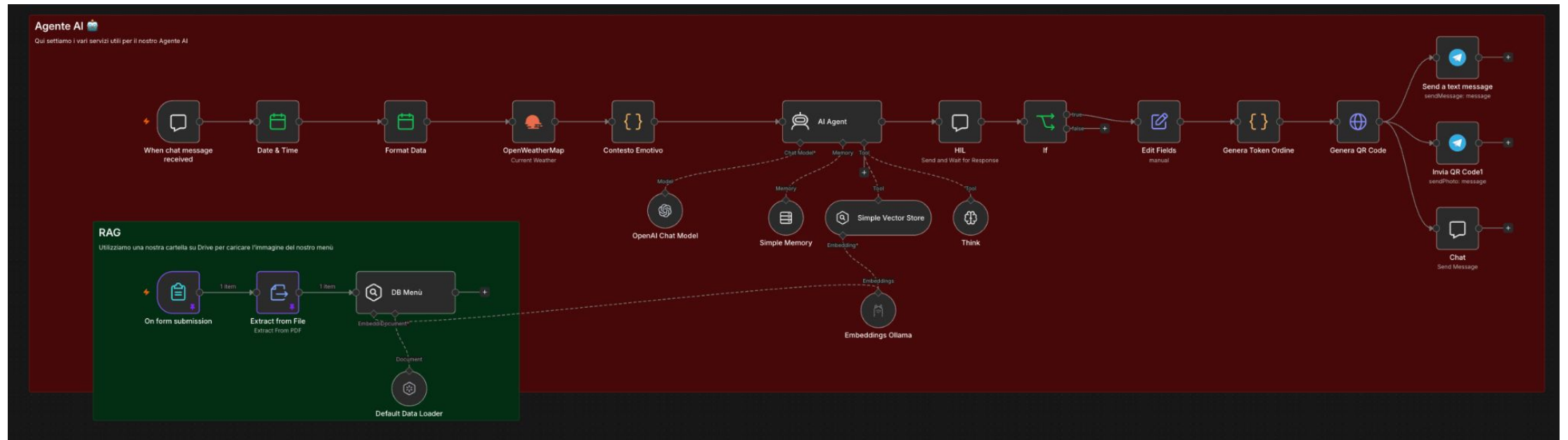
Riccardo Figliozzi

Teacher @Data Masters
Data Scientist / AI & Automation
Specialist
firenze.dev staff

Materiale



Un agente che consiglia la tigella perfetta



Cosa saprete fare alla fine del WS

- Importare e far girare TigellAI
- Capire ogni nodo del workflow
- Distinguere flusso RAG e chat
- Toccare un agente AI reale



Le 3 cose che ti servono oggi

- OpenAI API key
- [OpenWeatherMap](#) API key (il meteo)

Tanta tanta pazienza



Filosofia no-code/low-code

No-code

- Creazione di automazioni senza l'uso di linguaggi di programmazione
- Nodi già pronti per connettere servizi comuni (es. Gmail, Slack, Google Sheets)

Low-code

- Possibilità di personalizzare i workflow con espressioni o codice custom
- Maggiore flessibilità per scenari complessi

Filosofia → abbassare la barriera d'ingresso mantenendo la potenza del codice quando serve

Cos'è n8n?

- È una piattaforma di automazione basata su **workflow visivi** (flussi composti da nodi).
- Ha un approccio **No-code e Low-code**.
- È **Open-source** e permette di **connettere centinaia di servizi** comuni come Gmail, Outlook e Google Drive.



n8n Cloud vs Self Hosted



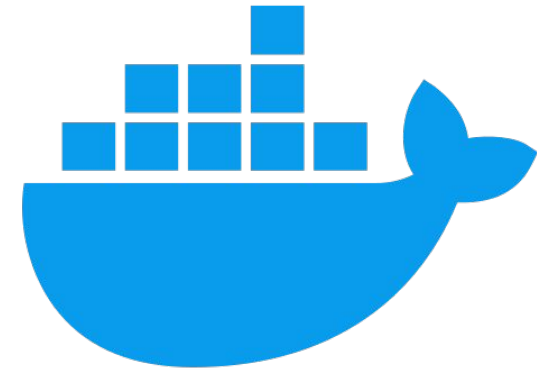
	Cloud	Self-Hosted
Pro	- Setup immediato - Aggiornamenti gestiti - Supporto incluso	- Controllo totale su dati - Scalabilità
Contro	- Costo mensile - Minore controllo	- Setup tecnico richiesto - Manutenzione a carico dell'utente - Supporto limitato alla community
Ideale per	Chi vuole iniziare subito senza configurazioni tecniche	Chi necessita di controllo totale e integrazione avanzata
Esempi di utilizzo	Startup, PMI, team non tecnici	Grandi aziende, sviluppatori, ambienti on-premise

n8n in locale

1. Assicurarsi di avere Docker Desktop installato ed in esecuzione.

Eseguire il seguente comando da terminale:

```
docker volume create n8n_data
docker run -it \
  --name n8n \
  -p 5678:5678 \
  -e GENERIC_TIMEZONE="Europe/Rome" \
  -e TZ="Europe/Rome" \
  -e N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true \
  -e N8N_RUNNERS_ENABLED=true \
  -v n8n_data:/home/node/.n8n \
  docker.n8n.io/n8nio/n8n
```



2. Una volta che l'installazione è completata, andare su:

<http://localhost:5678>

n8n in locale - PowerShell

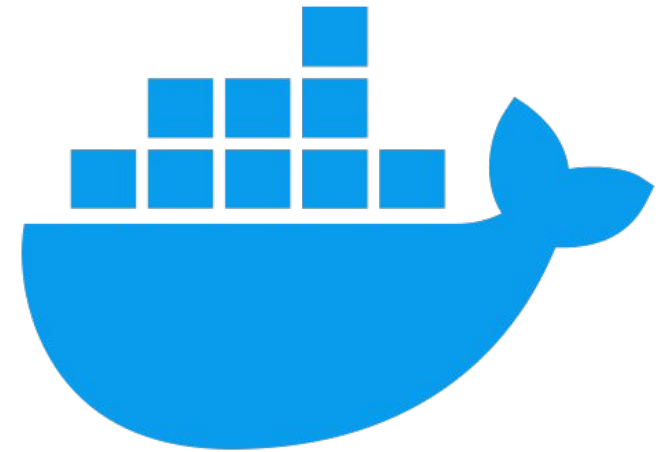
1. Assicurarsi di avere Docker Desktop installato ed in esecuzione.

Eseguire il seguente comando da terminale:

```
docker run -it --name n8n -p 5678:5678 -e GENERIC_TIMEZONE="Europe/Rome" -e TZ="Europe/Rome" -e  
N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true -e N8N_RUNNERS_ENABLED=true -v n8n_data:/home/node/.n8n  
docker.n8n.io/n8nio/n8n
```

2. Una volta che l'installazione è completata, andare su:

<http://localhost:5678>



Docker compose

Per una installazione più corretta e personalizzata in micorservizi:

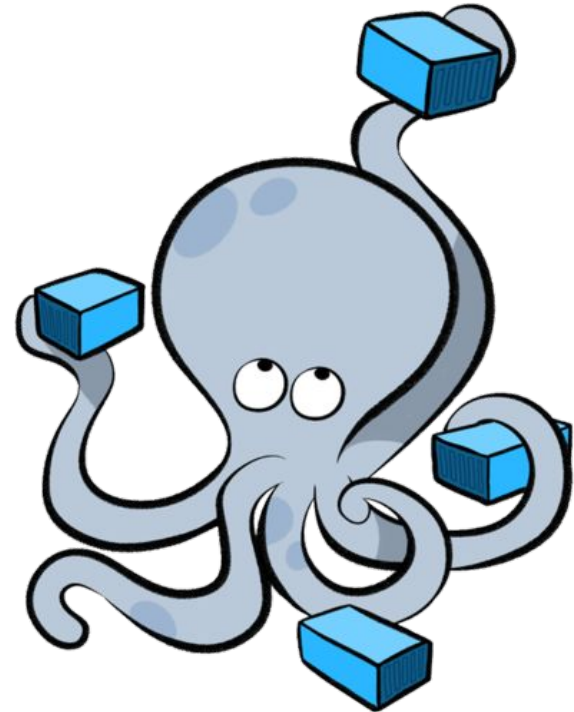
```
services:  
  n8n:  
    image: docker.n8n.io/n8nio/n8n:latest  
    restart: always  
    container_name: n8n-compose  
    ports:  
      - "127.0.0.1:5678:5678"
```

Eseguire il docker compose con il comando:

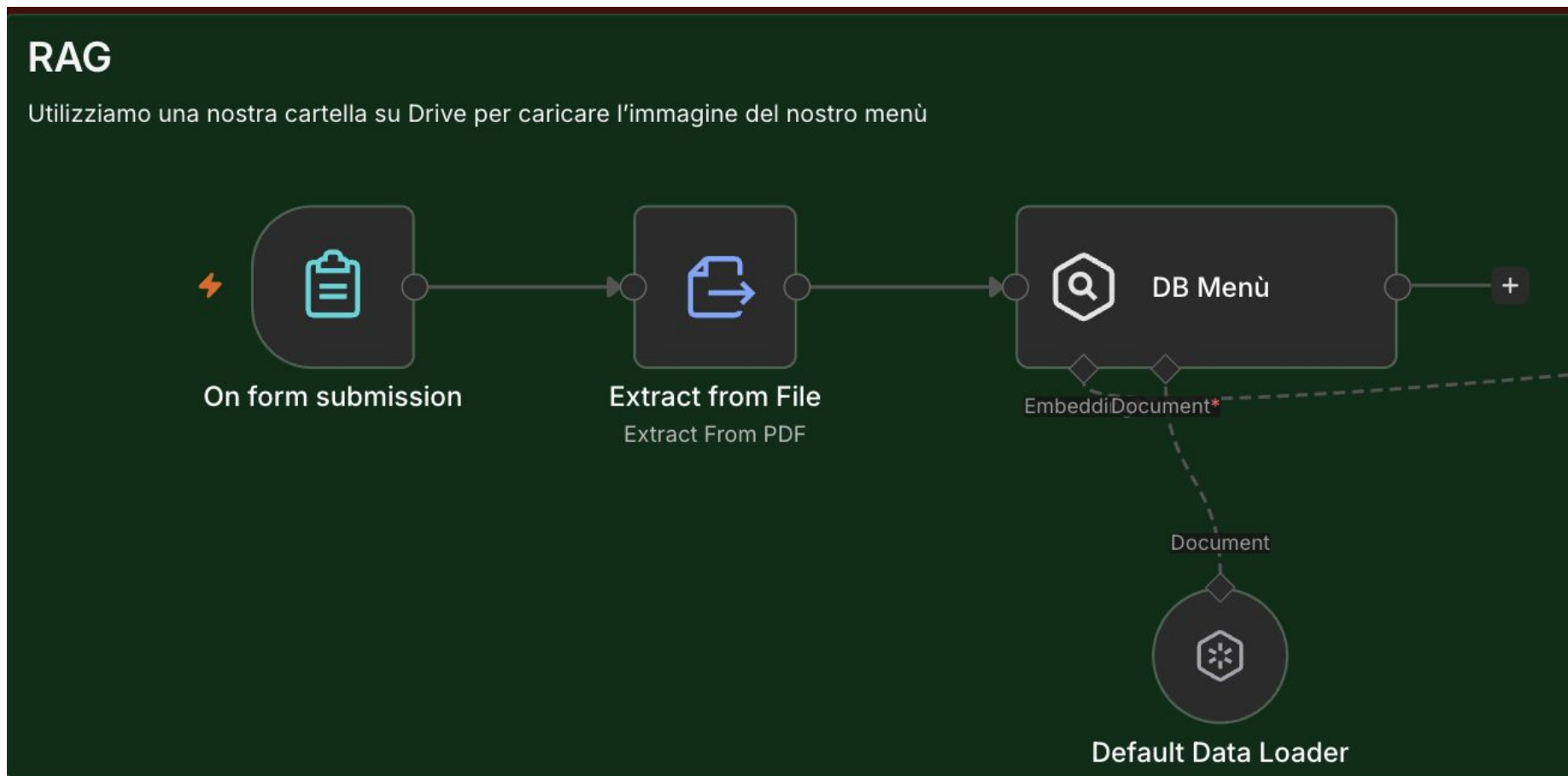
```
docker compose up
```

N.B.

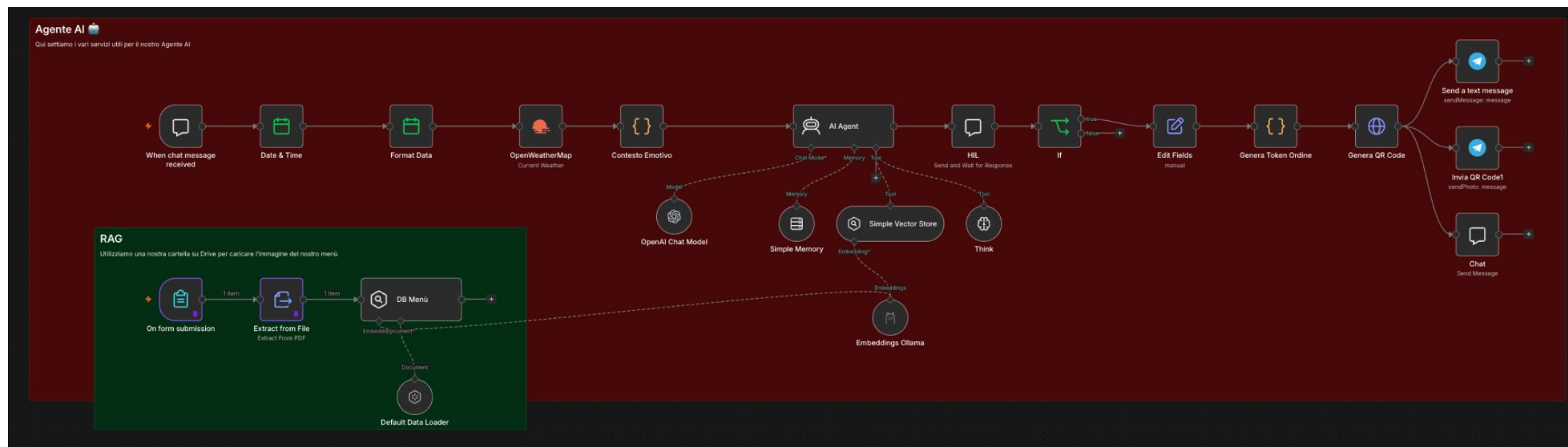
Per eseguire il comando bisogna creare una cartella con dentro un file 'compose.yml'



TigellAI vive di due flussi paralleli



TigellAI vive di due flussi paralleli

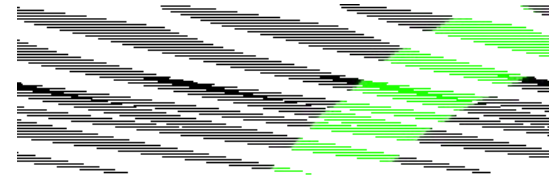


Agente con RAG vs agente senza

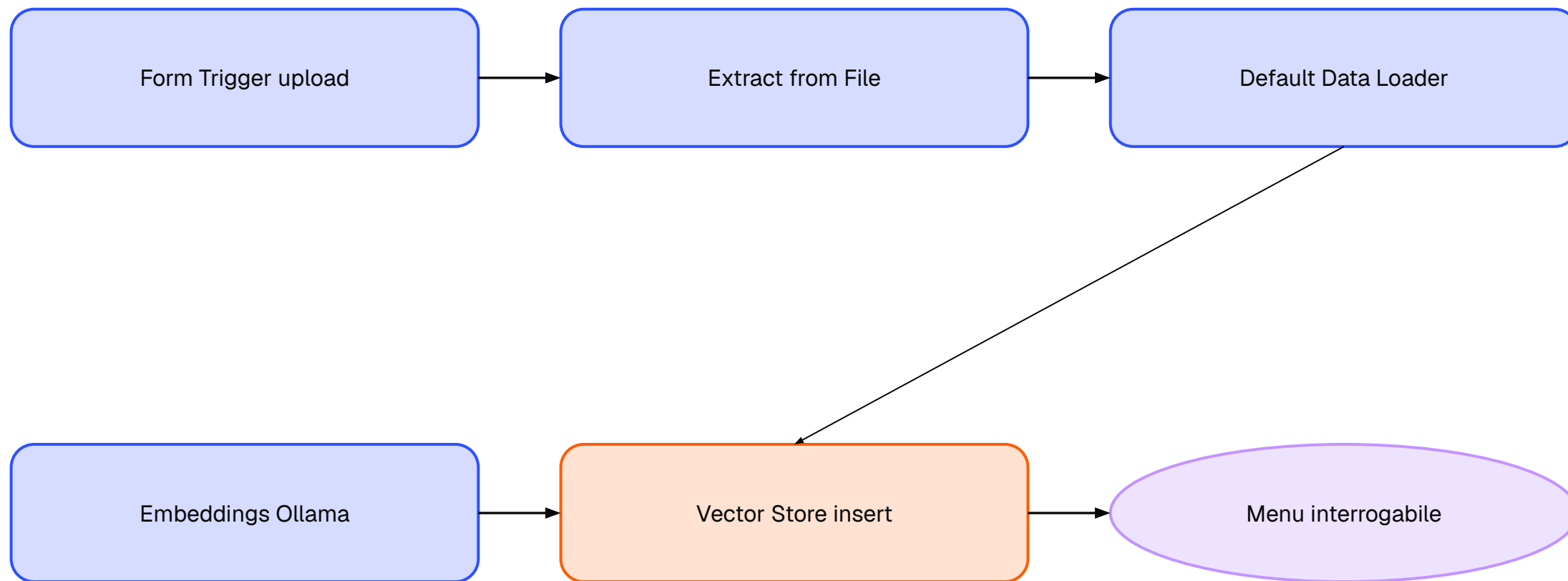
- Inventa piatti inesistenti
- Non conosce i prezzi veri



- Risponde dal menù reale
- Prezzi e ingredienti corretti

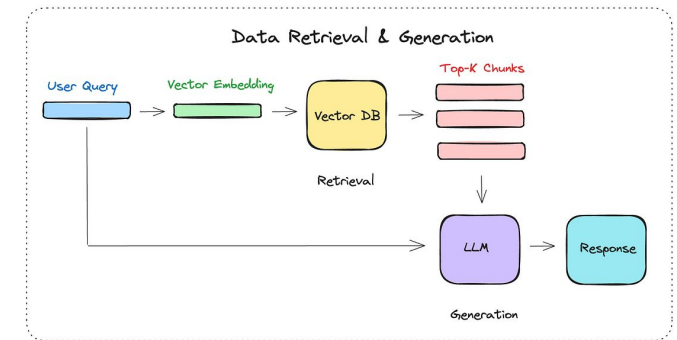
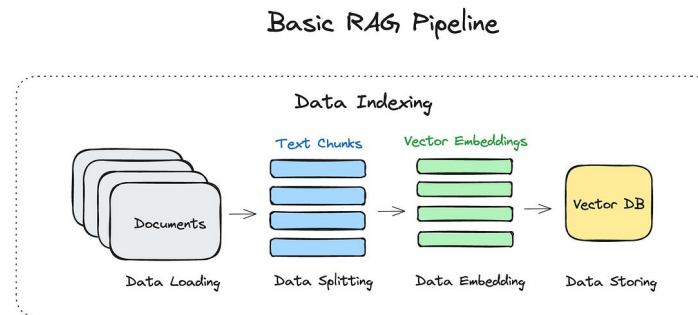


La pipeline che divora il PDF



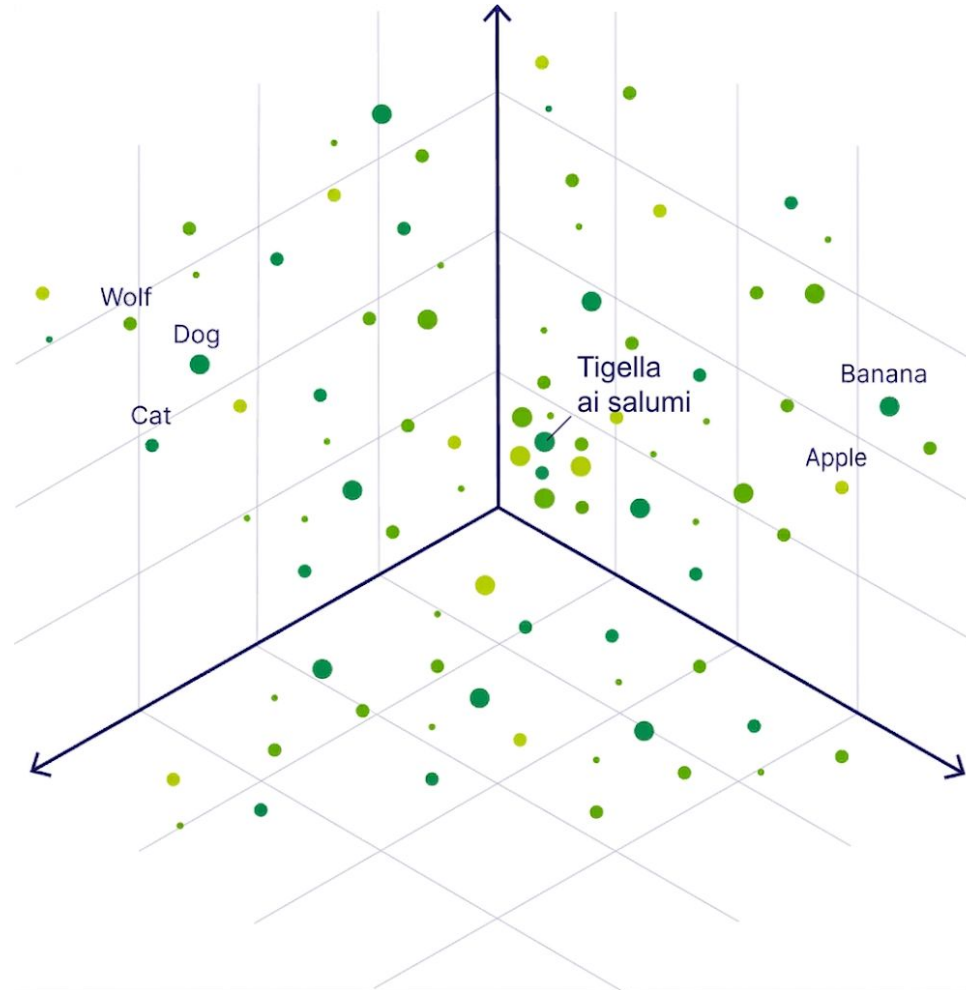
Cosa fa ciascun nodo RAG

- Form Trigger: carichi il PDF
- Extract: PDF diventa testo
- Data Loader: spezza in chunk
- Embeddings: testo diventa numeri
- Vector Store: archivia i vettori



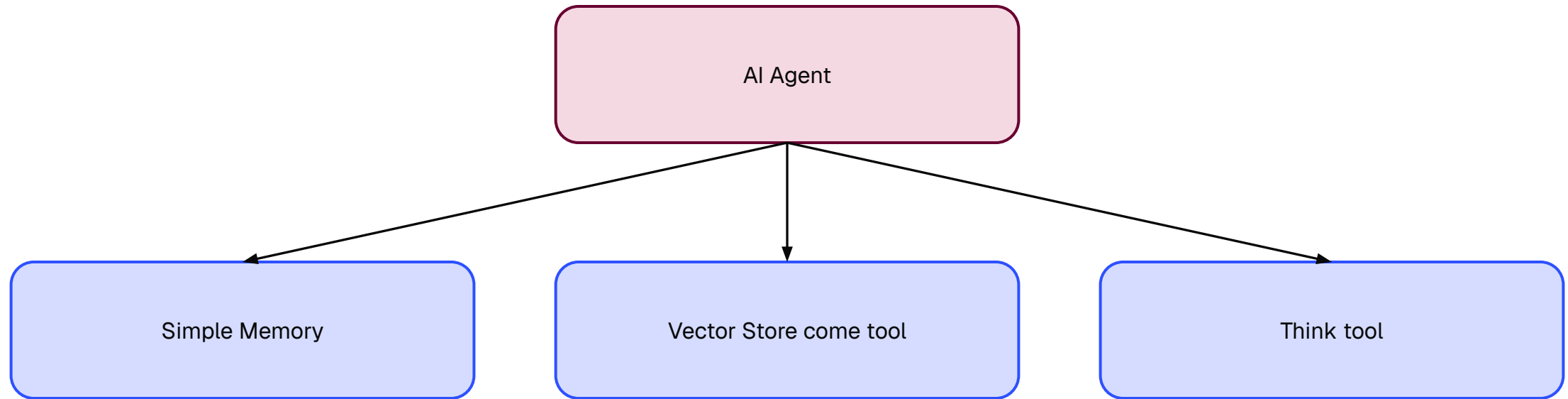
L'embedder è il traduttore

Trasforma 'tigella ai salumi'
in un punto preciso dello
spazio vettoriale



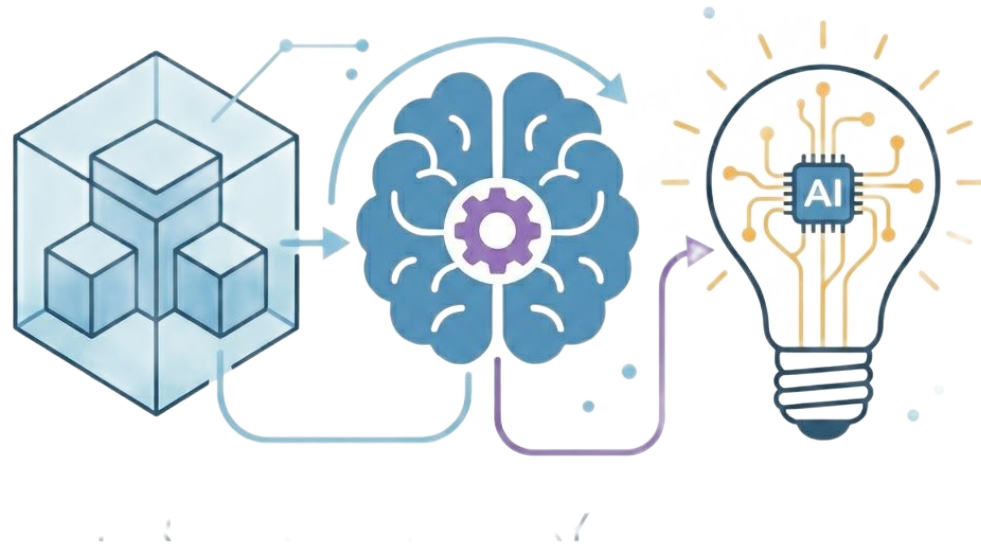
Cosa c'è dentro l'AI Agent

L'agente non è solo il modello: è modello + memoria + strumenti



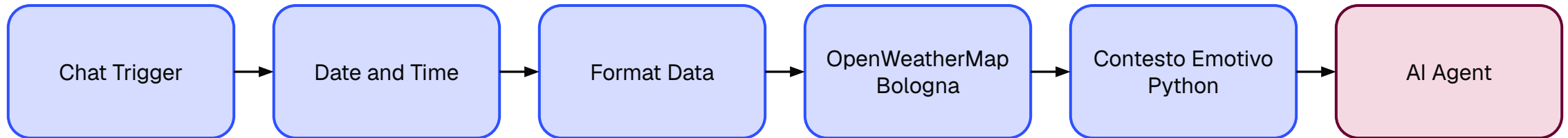
I tre strumenti dell'agente

1. **Vector Store**: consulta il menù
2. **Memory**: ricorda cosa ha detto
3. **Think**: ragiona prima di parlare



Il contesto prima della risposta

Ora, meteo e mood arrivano all'agente prima che apra bocca



Perché iniettare ora e meteo

Alle 23 sotto la pioggia consiglio diverso che a mezzogiorno col sole.



Contesto Emotivo: ora che diventa mood

```
from datetime import datetime

items = _items
item = items[0]

ora = datetime.now()
h = ora.hour
giorno = ora.weekday() # 0 = lunedì, 6 = domenica

giorni = ["lunedì", "martedì", "mercoledì", "giovedì", "venerdì", "sabato", "domenica"]
giorno_nome = giorni[giorno]
is_weekend = giorno >= 5

# Mappa ogni slot orario su un momento emotivo + mood
if 0 <= h < 6:
    momento = "notte fonda"
    mood = "ha bisogno di massimo conforto, qualcosa di caldo e avvolgente che lo consoli"
elif 6 <= h < 9:
    if is_weekend:
        momento = "mattina del weekend"
        mood = "si sveglia senza fretta, vuole coccolarsi con qualcosa di ricco senza sensi di colpa"
    else:
        momento = "mattina presto"
        mood = "ha fretta e ha bisogno di energia per iniziare la giornata lavorativa"
elif 9 <= h < 12:
    momento = "mattinata"
    mood = "si concede una pausa, vuole qualcosa di soddisfacente ma non troppo pesante"
elif 12 <= h < 14:
    momento = "pranzo"
    mood = "ha poco tempo, cerca qualcosa di veloce, sostanzioso e senza fronzoli"
elif 14 <= h < 17:
    momento = "pomeriggio"
    mood = "non ha una vera fame, cerca uno sfizio o qualcosa di insolito per spezzare la giornata"
elif 17 <= h < 20:
    momento = "aperitivo"
    mood = "e in mood sociale, probabilmente non e solo e vuole qualcosa da condividere e da accompagnare con una bevanda"
elif 20 <= h < 23:
    momento = "cena"
    mood = "si e guadagnato qualcosa di speciale dopo una lunga giornata, il prezzo conta meno del piacere"
else:
    momento = "tarda serata"
    mood = "e stanco ma irrequieto, cerca il comfort food definitivo, qualcosa che lo faccia sentire a casa"

# Costruisce la stringa di contesto emotivo
contesto_emotivo = f"{momento} di {giorno_nome}. Il cliente probabilmente {mood}."

# Aggiunge contesto extra per il weekend
if is_weekend and 11 <= h < 15:
    contesto_emotivo += " E weekend: c'e piu tempo, meno stress e maggiore apertura a sperimentare gusti nuovi."
elif is_weekend and h >= 19:
    contesto_emotivo += " Serata del weekend: atmosfera rilassata, voglia di stare bene e di non avere rimpianti."

# Restituisce tutti i campi precedenti (meteo, ecc.) piu il nuovo contesto
return [{
    "json": {
        **item,
        "contesto_emotivo": contesto_emotivo,
        "momento_nome": momento,
        "giorno_settimana": giorno_nome,
        "ora_corrente": h
    }
}]
```

Modelli Single-Call

- Un'esecuzione diretta del modello per una *singola* task ben definita.
- **Input:** Un prompt specifico (la richiesta) e i dati di contesto (es. il testo da riassumere).
- **Output:** Un testo generato (riassunto, traduzione, descrizione) o una classificazione.

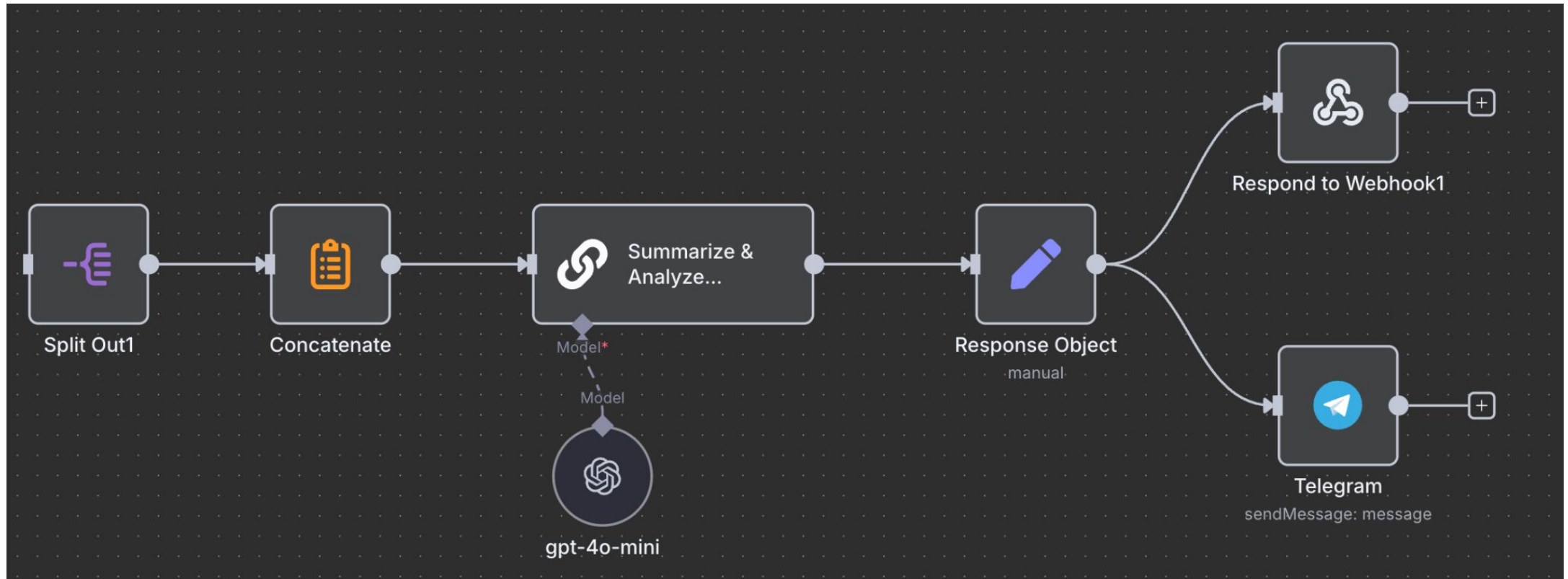
Modelli Single-Call

Limiti principali:

- **Non ha memoria:** Non ricorda le interazioni precedenti (a meno che tu non la fornisca nel prompt).
- **Non agisce:** Non può cercare informazioni su internet, inviare email o aggiornare un database da solo.

Ruolo nel Workflow: È un **passo di trasformazione dati**, come un nodo **Code** o **Set**, ma con capacità generativa.

Modelli Single-Call



AI Agent

Un AI Agent è un sistema che utilizza un LLM per la **ragione** e il **processo decisionale**.
Deciderà, passo dopo passo, la sequenza di azioni necessarie per completare un obiettivo.

Il processo:

1. Riceve l'**obiettivo** (Goal).
2. Analizza l'obiettivo e decide se e quali **Strumenti (Tools)** utilizzare.
3. Possiede una memoria.
4. Esegue lo strumento (es. cerca sul Web).
5. Valuta il risultato dello strumento.
6. Ripete il ciclo finché l'obiettivo non è completato o raggiunge il limite.

AI Agent

System prompt

Sei un esperto bolognese di tigelle con un dono speciale: senti l'umore delle persone e scegli per loro la tigella perfetta.

In base alla richiesta del cliente, alla temperatura, al meteo, al vento, all'ora e soprattutto al CONTESTO EMOTIVO del momento, devi:

1. Scegliere la farcitura della tigella piu appropriata
2. Abbinare una bevanda coerente con il momento della giornata
3. Suggestire un contorno adatto
4. Dare il prezzo totale

Il contesto emotivo e il tuo segreto: usalo per calibrare il tono della risposta e la scelta degli ingredienti. Se e aperitivo suggerisci qualcosa di condivisibile. Se e notte fonda punta sul conforto. Se e pranzo veloce vai sul pratico.

Prima di rispondere ****controlla all'interno della memoria**** se ci sono delle risposte del cliente ('Human') e modifica l'output secondo le sue richieste.

Utilizza SEMPRE i seguenti tools:

- Vector Store Retriever: ti fornisce il menu delle tigelle disponibili.

Rispondi solo con il consiglio, in modo caldo e personale. Non includere query fatte al vector store.

AI Agent

Prompt Message

- Richiesta del cliente: `{{ $('When chat message received').item.json.chatInput }}`
- Temperatura: `{{ $('OpenWeatherMap').item.json.main.temp }}`
- Descrizione: `{{ $('OpenWeatherMap').item.json.weather[0].description }}`
- Vento: `{{ $('OpenWeatherMap').item.json.wind.speed }}`
- Ora: `{{ $('Date & Time').item.json.currentDate }}`
- Contesto emotivo: `{{ $json.contesto_emotivo }}{{ $('Contesto Emotivo').item.json.contesto_emotivo }}`

Tools

Gli Strumenti sono le **funzioni esterne** che l'Agent può invocare. Sono le "API" che l'LLM decide di chiamare in autonomia.

Esempi di Tools in n8n:

- **Web Search:** Ricercare informazioni aggiornate online.
- **API/HTTP Request:** Interagire con sistemi esterni (es. CRM, Database).
- **Code Execution:** Eseguire codice Python o JavaScript per calcoli complessi.
- **Execute Workflow:** Richiamare un altro Sub-Workflow n8n come se fosse uno Strumento.

Single-Call Vs AI Agent

	Modello Single-Call	AI Agent
Obiettivo: Ottenere la quotazione azionaria di Tesla, scrivere un breve riassunto e inviarlo.	1. HTTP Request (API Azioni) 2. LLM Node (Riassunto) 3. Email Node (Invio)	1. Agent Node (Riceve: "Invia un riassunto della quotazione Tesla") 2. L'Agent decide: a) Usa Stock Price Tool (API) b) Usa la sua intelligenza per riassumere c) Usa Email Tool (Invia).
Pianificazione:	Fissa e definita dall'utente.	Dinamica, decisa dall'AI.
Flessibilità:	Bassa. Se la API fallisce, il processo si interrompe.	Alta. Può tentare strategie diverse (es. cercare su un altro sito se il primo fallisce).

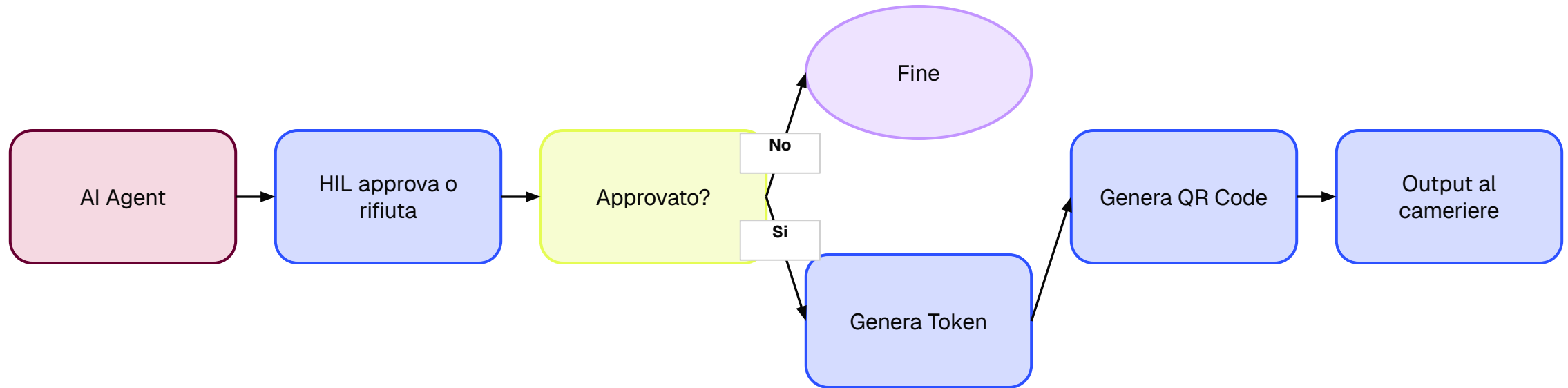
Perché un umano nel mezzo

L'agente suggerisce, ma nessun ordine parte senza un sì esplicito del cliente.



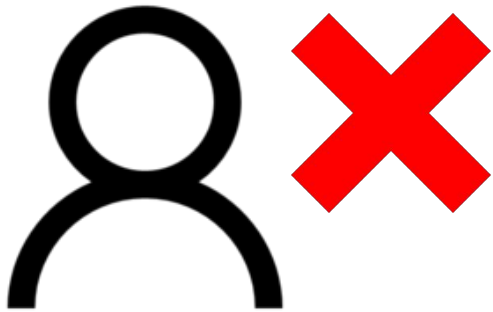
Dal suggerimento alla cucina

Il branch If è il bivio: si continua solo col consenso

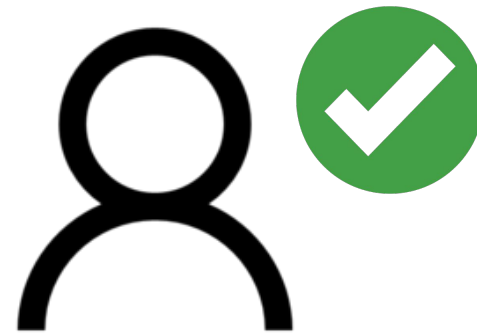


Con HIL vs senza HIL

- L'agente ordina da solo
- Errori arrivano in cucina
- Zero controllo del cliente



- Il cliente conferma l'ordine
- Errori bloccati prima
- Fiducia e tracciabilità



Token ordine: dal testo al QR

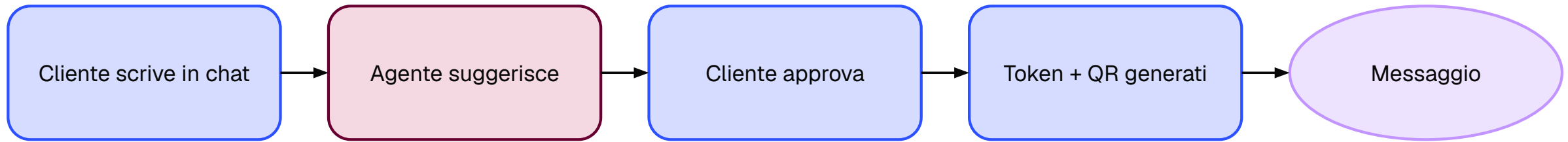
```
import base64, hashlib, json

payload = json.dumps(ordine, ensure_ascii=False)
token = base64.urlsafe_b64encode(payload.encode()).decode()
codice_breve = hashlib.sha256(token.encode()).hexdigest()[:8]

url_qr = (
    "https://api.qrserver.com/v1/create-qr-code/"
    f"?size=300x300&data={codice_breve}"
)

return [{"token": token, "codice": codice_breve, "qr": url_qr}]
```

Un ordine, dall'inizio alla fine



Se qualcosa si rompe??

- Leggi il nodo rosso per primo
- Controlla le credenziali API
- Vector Store vuoto? Ricarica PDF
- Webhook spento? Riattiva trigger

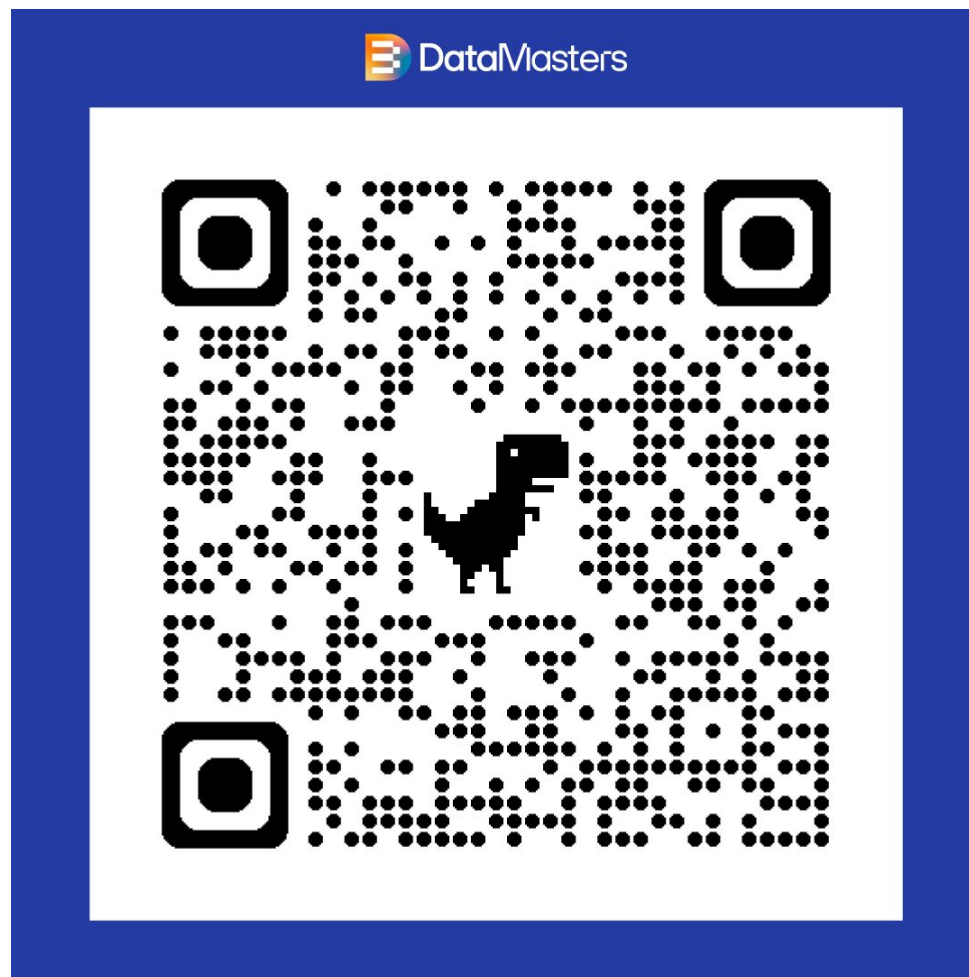


5 idee da non dimenticare

- RAG dà fonti vere all'agente
- L'agente è un LLM con tools
- Il contesto cambia la risposta
- L'umano resta nel loop
- n8n collega tutto visualmente



Grazie e buone tigelle!



Demande?



DataMasters